INDIRA GANDHI NATIONAL OPEN UNIVERSITY

School of Computer and Information Sciences (SOCIS)

Maidan Garhi, New Delhi - 110068

MAJOR PROJECT REPORT

SMART MEDICAL APPOINTMENT & RISK PREDICTION SYSTEM

A Full-Stack Enterprise Healthcare Scheduling & Automated Symptom Triage Platform

Project Report submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF COMPUTER APPLICATIONS (BCA)

Course Code: BCSP-064 (Major Project)

SUBMITTED BY

Candidate Name: Preeti
Enrollment Number: 2401048429
Programme: BCA (Semester VI)
Study Centre: IGNOU Regional Centre

UNDER THE GUIDANCE OF

Project Guide: Gaurav Dhyani
Designation: Cloud Administrator
Academic Institution: SOCIS, IGNOU
City: New Delhi, India

Academic Session: 2025 - 2026

CERTIFICATE OF ORIGINALITY

This is to certify that the project report entitled **"Smart Medical Appointment and Risk Prediction System"**, submitted by **Preeti** (Enrollment Number: **2401048429**) in partial fulfillment of the requirements for the award of the degree of **Bachelor of Computer Applications (BCA)** from **Indira Gandhi National Open University (IGNOU)**, is an authentic and original work carried out by her under my guidance and supervision.

To the best of my knowledge, the matter embodied in this project report has not been submitted by the candidate or any other person for the award of any other degree or diploma to any university or institution.

Preeti
Enrollment No: 2401048429
BCA Final Year, IGNOU
(Signature of the Candidate)
Date: _____

Gaurav Dhyani
Designation: Cloud Administrator
Project Guide, IGNOU
(Signature of the Guide)
Date: _____

ACKNOWLEDGEMENT

I would like to express my profound gratitude and sincere appreciation to my esteemed project guide, **Gaurav Dhyani** (Cloud Administrator), for his invaluable guidance, continuous encouragement, constructive critiques, and technical insights throughout the design, implementation, and documentation of the **Smart Medical Appointment and Risk Prediction System**. His expertise in enterprise software architectures, cloud standards, and system security has been instrumental in shaping this project to academic and professional excellence.

I also extend my heartfelt thanks to the faculty members, coordinators, and administrative staff of the **School of Computer and Information Sciences (SOCIS), IGNOU**, for providing a comprehensive curriculum that laid a solid foundation in software engineering, database normalization, web architectures, and object-oriented paradigms.

Lastly, I express my deepest love and gratitude to my family and friends for their steadfast encouragement, moral support, and patience throughout the tenure of my BCA programme.

Preeti
Enrollment Number: 2401048429
Bachelor of Computer Applications (BCA)
Indira Gandhi National Open University (IGNOU)
Place: New Delhi
Date: _____

TABLE OF CONTENTS

1. CHAPTER 1: INTRODUCTION **1**
1.1 Project Overview 1
1.2 Purpose and Objectives 1
1.3 Project Category & WDLC (Web Development Life Cycle) 2
1.4 Scope & Limitations of the Existing System 2
1.5 Hardware and Software Environment 3
2. CHAPTER 2: SYSTEM ANALYSIS & REQUIREMENTS **4**
2.1 Feasibility Study (Technical, Operational, Economic) 4
2.2 Software Requirement Specification (SRS) 4
2.3 User Role Identification (Admin, Doctor, Patient) 5
2.4 Functional & Non-Functional Requirements 5
3. CHAPTER 3: SYSTEM DESIGN & UML / DFD MODELING **6**
3.1 3-Tier Layered Architecture 6
3.2 Use Case Diagram 7
3.3 Class Diagram 8
3.4 Data Flow Diagram: Level 0 (Context Diagram) 9
3.5 Data Flow Diagram: Level 1 10
3.6 Data Flow Diagram: Level 2 (Detailed Functional Decomposition) 11
3.7 Entity-Relationship (ER) Diagram 12
3.8 Database Design & Normalized Relational Tables (3NF) 13
4. CHAPTER 4: IMPLEMENTATION DETAILS & ALGORITHMS **15**
4.1 Technology Stack & Architectural Patterns 15
4.2 Module-Wise Functional Description (Modules 1-9) 15
4.3 Rule-Based Risk Prediction Engine Logic 16
4.4 Seamless Triage-to-Booking Integration Flow 16
4.5 Security Architecture & Stateless JWT Lifecycle 16
5. CHAPTER 5: VERIFICATION & TESTING **17**
5.1 Testing Strategy & Methodology 17
5.2 Synopsis Test Case Execution Table (TC-01 to TC-05) 17
5.3 Automated Unit & Integration Testing Matrix (21/21 Passed) 18
5.4 Defect Resolution & Concurrency Safeguards 19
6. CHAPTER 6: USER GUIDE & SYSTEM OPERATION **20**
6.1 Application Startup & Dev Profile Configuration 20
6.2 Pre-Loaded Demonstration Accounts 20
6.3 Patient, Doctor & Administrator Portals 20
7. CHAPTER 7: CONCLUSION & FUTURE SCOPE **21**
7.1 Conclusion & Synopsis Compliance 21
7.2 Future Scope 21
8. CHAPTER 8: IGNOU VIVA VOCE PREPARATION GUIDE **22**

CHAPTER 1

INTRODUCTION

1.1 Project Overview

In modern healthcare facilities, managing patient appointments manually leads to prolonged waiting queues, scheduling errors, lost records, and patient frustration. Furthermore, patients often delay seeking medical attention due to a lack of early awareness regarding subtle symptoms.

The **Smart Medical Appointment and Risk Prediction System** is a computerized web application developed to bridge this gap. Built using Java 17 and Spring Boot 3, it provides automated conflict-free doctor appointment scheduling alongside an automated **Rule-Based Risk Prediction Engine** that triages patient symptoms to promote early health awareness and guide timely clinical consultations.

1.2 Purpose and Objectives

- **Digital Scheduling:** Eliminate physical OPD queues through computerized time-slot allocation.
- **Double-Booking Prevention:** Enforce transactional database locks and slot availability validation.
- **Symptom Triage:** Provide an automated rule-based risk prediction engine categorizing health urgency into LOW, MEDIUM, or HIGH risk.
- **Triage-to-Consultation Integration:** Automatically pre-fill doctor consultation reasons for patients with high or moderate risk scores.
- **Clinical Records:** Provide doctors with patient clinical profiles (Blood Group, Medical History) and consultation notes/e-prescription documentation tools.
- **Operational Traceability:** Provide administrators with real-time analytics, exportable CSV reports, security audit trails, and portable SQL database backups.

1.3 Project Category & Web Development Life Cycle (WDLC)

The project falls under the category of **Enterprise Web-Based Applications**. It was engineered following the **Web Development Life Cycle (WDLC)**:

1. **Requirement Analysis:** Understanding outpatient workflows, doctor availability patterns, and clinical symptom rules.
2. **Planning:** Defining the technology stack (Spring Boot 3, MySQL/H2, Bootstrap 5) and timeline.
3. **Design:** Creating architectural blueprints, UML diagrams, DFDs (Level 0, 1, 2), and normalized 3NF schemas.
4. **Development:** Writing clean backend REST controllers, service components, repositories, and responsive front-end views.
5. **Testing:** Verifying system behavior using JUnit 5, Mockito, and Postman API test suites.
6. **Deployment:** Packaging into a runnable JAR using Maven and hosting with embedded Tomcat.
7. **Maintenance:** Establishing automated audit logging, database backup exports, and modular services for future scalability.

1.4 Scope & Limitations of the Existing System

Drawbacks of Traditional / Manual Systems:

- Long waiting times and crowded physical reception queues.
- Manual paper records prone to loss, wear, and misplacement.
- High risk of double-booking due to uncoordinated scheduling across multiple receptionists.
- Complete absence of preliminary symptom risk awareness.
- Time-consuming manual report compilation.

Scope of Proposed System:

- Covers outpatient appointment booking, rescheduling, and cancellation.
- Provides dedicated portals for Patients, Doctors, and Hospital Administrators.
- Incorporates deterministic health risk assessment based on 10 symptom indicators.
- Provides complete disaster recovery database backup capabilities.

1.5 Hardware & Software Environment

Component	Specification / Technology
Operating System	Windows 10 / 11, Linux, macOS
Processor	1.6 GHz or faster 64-bit multi-core CPU
RAM	4 GB minimum (8 GB recommended)
Disk Space	10 GB available storage
Backend Framework	Spring Boot 3.3.4, Java 17 LTS
Security	Spring Security 6 with stateless JWT (io.jsonwebtoken 0.11.5) & BCrypt
Database	MySQL 8.x (Production) / H2 in-memory (MySQL Compatibility Mode)
Frontend	HTML5, CSS3, JavaScript (ES6+ Fetch API), Bootstrap 5.3.3
Build Tool	Apache Maven 3.9+ (Wrapper mvnw.cmd included)
API Testing	Postman 2.1 Collection & JUnit 5 Surefire Suite

CHAPTER 2

SYSTEM ANALYSIS & REQUIREMENTS

2.1 Feasibility Study

1. **Technical Feasibility:** Developed using industry-standard enterprise Java and Spring Boot 3, providing high performance, robust concurrency management, and seamless JPA ORM mapping. The architecture is fully portable across Windows and Linux environments.
2. **Operational Feasibility:** The role-oriented user interface is responsive, clean, and requires zero technical training for patients or healthcare staff. Intuitive cards and clear color coding simplify daily operations.
3. **Economic Feasibility:** Built entirely on open-source frameworks, runtime engines, and development tools, incurring zero software licensing expenses for the hospital.

2.2 Software Requirement Specification (SRS)

Functional Requirements:

- Secure authentication and role-based access control (RBAC) across Admin, Doctor, and Patient personas.
- Patient clinical profile management including Blood Group and Medical History tracking.
- Doctor availability schedule management with slot creation and safe deletion.
- Conflict-free appointment booking, atomic rescheduling, and cancellation with slot release.
- Rule-based health risk prediction analyzing 10 key symptom factors.
- Printable medical token slips, CSV analytics downloads, and SQL database backups.

Non-Functional Requirements:

- **Performance:** Sub-second API response times under standard outpatient loads.
- **Security:** BCrypt password hashing and signed JWT bearer tokens protecting all REST endpoints.
- **Reliability:** ACID-compliant relational transactions protecting against slot race conditions.
- **Maintainability:** Clean layered architecture separating Presentation, Business Logic, and Data layers.

2.3 User Role Identification

Role	Access Level	Primary Responsibilities
Patient	Self-Service Client Portal	Register profile, update clinical details, evaluate symptom risks, book appointments, reschedule, download token slips.
Doctor	Clinical Practitioner Portal	View daily consultation queue, add/delete availability slots, record clinical diagnosis notes, issue e-prescriptions.
Administrator	Master Hospital Management	Onboard/modify doctor accounts, oversee patient directories, export CSV audit reports, generate SQL database backups.

CHAPTER 3

SYSTEM DESIGN & UML / DFD MODELING

3.1 3-Tier Layered Architecture

The system follows a strict 3-tier enterprise architecture ensuring separation of concerns:

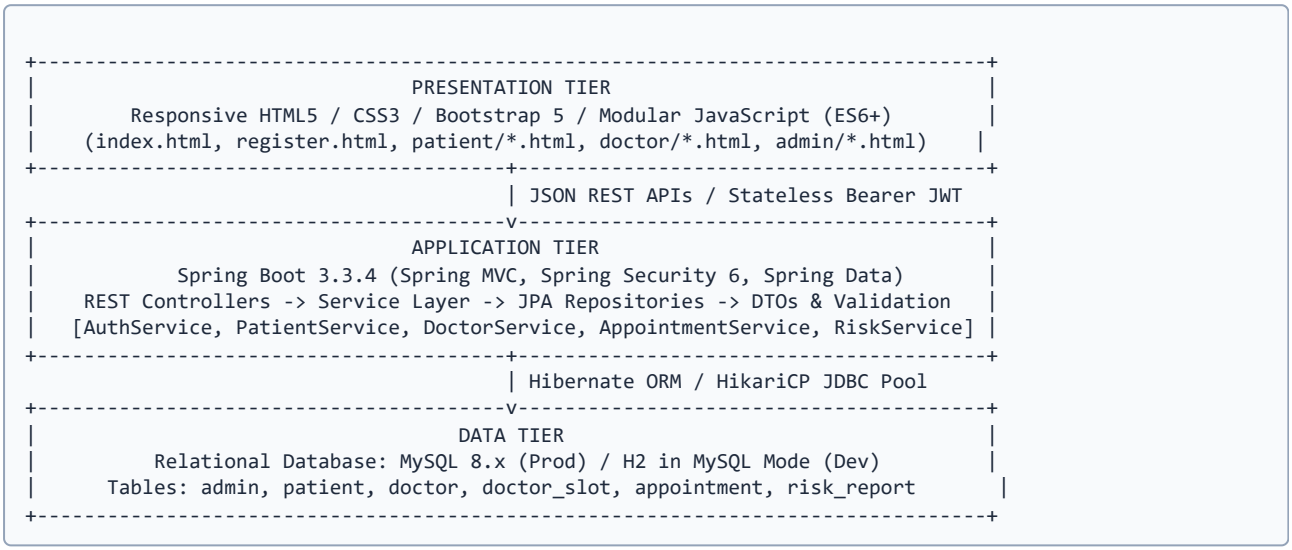


Figure 3.1: 3-Tier Layered Architecture Blueprint

3.2 Use Case Diagram

The Use Case Diagram depicts the system actors (Administrator, Patient, Doctor) and their functional boundaries:

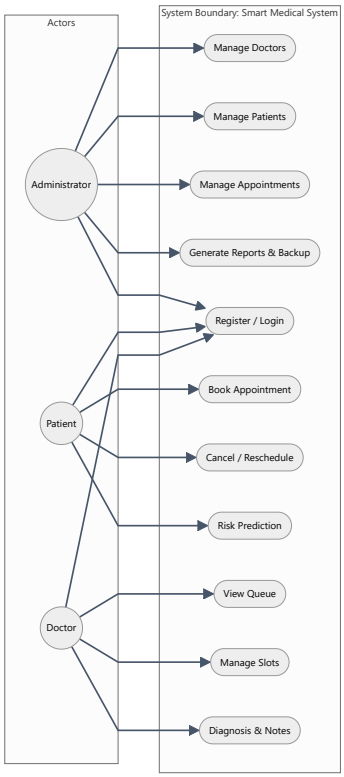


Figure 3.2: Unified System Use Case Diagram

3.3 Class Diagram

The Class Diagram illustrates domain entities, field attributes, operations, and structural associations:

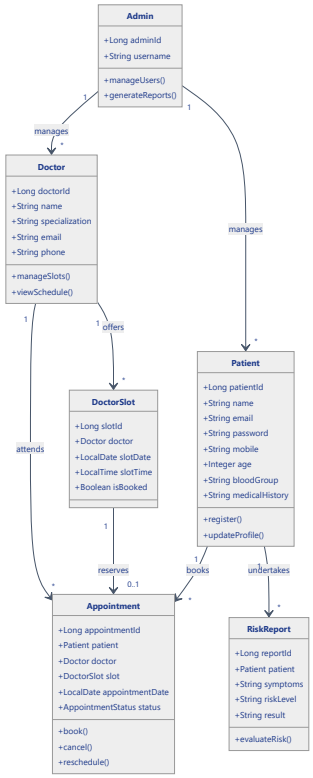


Figure 3.3: System Class Diagram

3.4 Data Flow Diagram: Level 0 (Context Diagram)

The Level 0 DFD defines the boundary between external entities and the centralized medical management system:

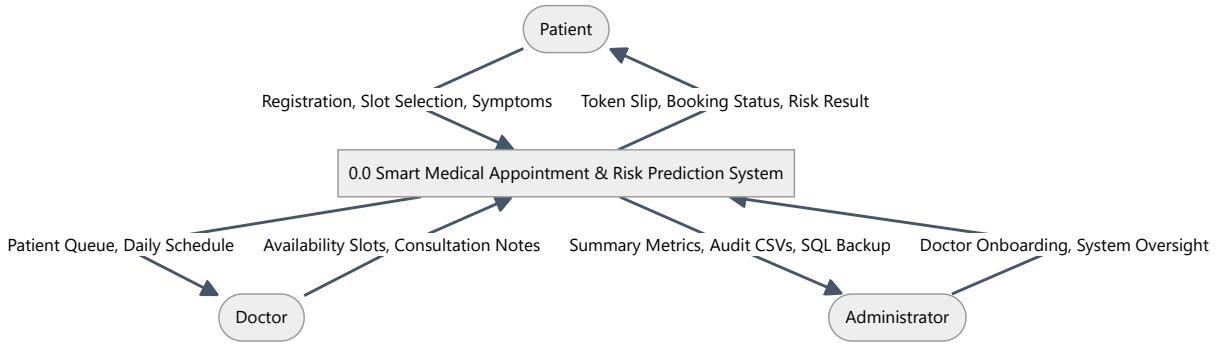


Figure 3.4: Data Flow Diagram - Level 0 (Context Diagram)

3.5 Data Flow Diagram: Level 1

The Level 1 DFD decomposes the system into 5 primary processes interacting with normalized datastores:

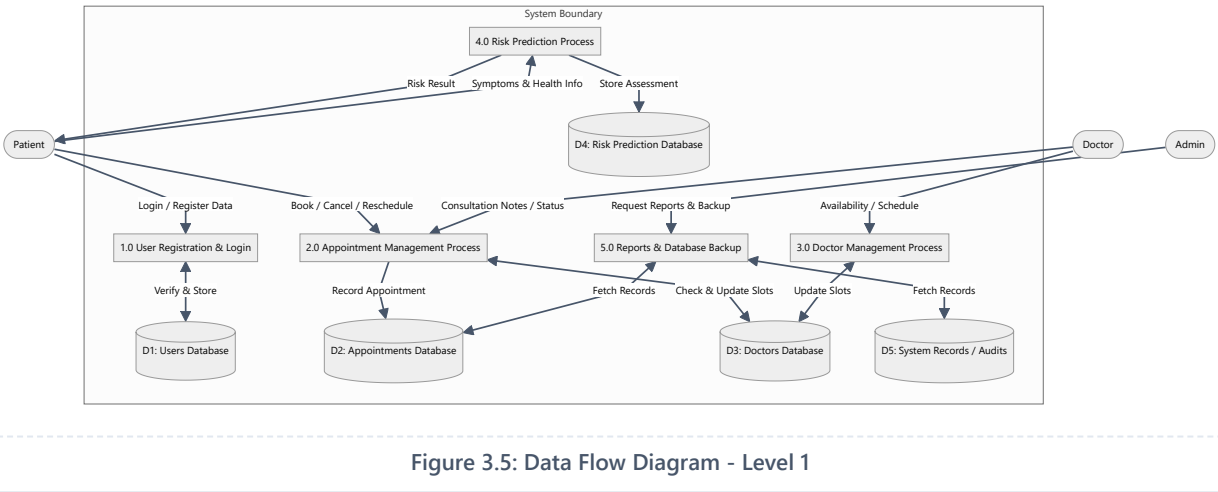


Figure 3.5: Data Flow Diagram - Level 1

3.6 Data Flow Diagram: Level 2 (Detailed Functional Decomposition)

Level 2 decomposes the system into 11 sub-processes (3.1 to 3.11) with data stores D1 to D4 as per the approved synopsis:

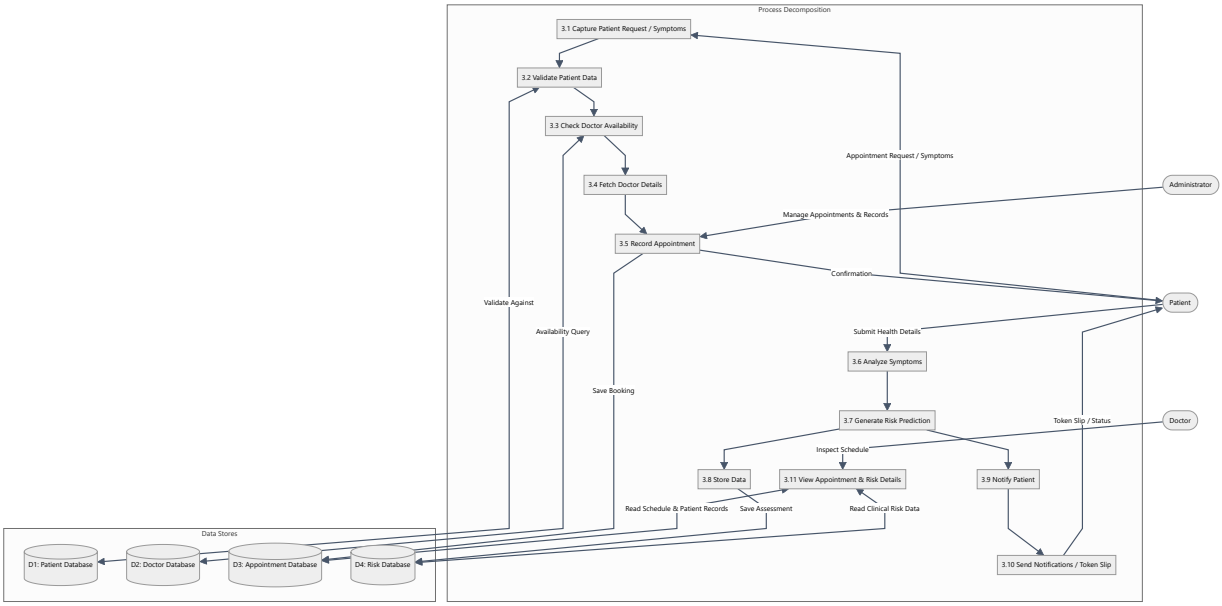


Figure 3.6: Data Flow Diagram - Level 2 (Functional Decomposition)

3.7 Entity-Relationship (ER) Diagram

The Entity-Relationship model shows tables, primary/foreign keys, and relational cardinality:

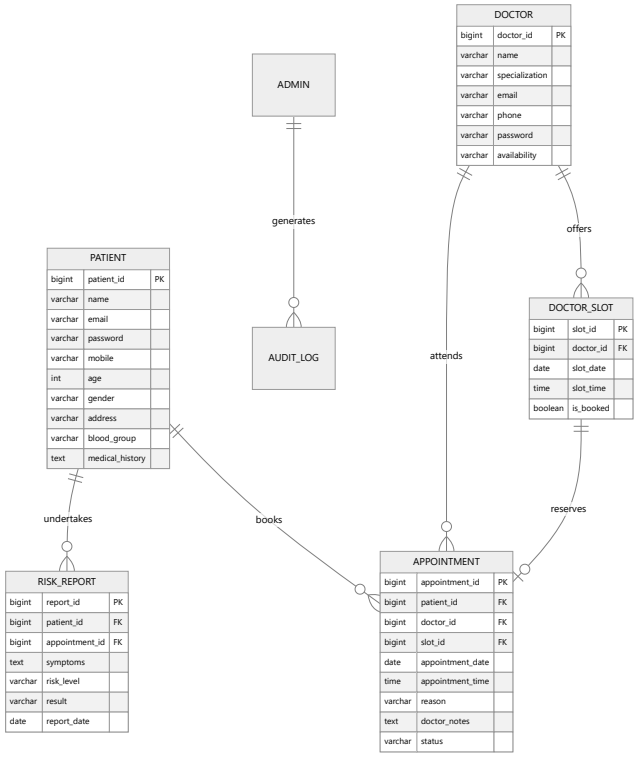


Figure 3.7: Entity-Relationship (ER) Diagram

3.8 Database Design & Normalized Relational Tables (3NF)

1. Admin Table

Stores master administrative credentials and privileges.

Field Name	Data Type	Constraints	Description
admin_id	BIGINT	PK, AUTO_INCREMENT	Unique administrator identifier
username	VARCHAR(50)	NOT NULL, UNIQUE	Administrative login username
password	VARCHAR(255)	NOT NULL	BCrypt salted password hash
email	VARCHAR(100)	NOT NULL, UNIQUE	Official administrative email
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Account creation timestamp

2. Patient Table

Stores patient identity, contact details, and clinical profile.

Field Name	Data Type	Constraints	Description
patient_id	BIGINT	PK, AUTO_INCREMENT	Unique patient identifier
name	VARCHAR(100)	NOT NULL	Full name of patient
email	VARCHAR(100)	NOT NULL, UNIQUE	Login email address
password	VARCHAR(255)	NOT NULL	BCrypt salted password hash
mobile	VARCHAR(15)	NOT NULL	Contact telephone number
age	INT	NOT NULL	Age in years
gender	VARCHAR(10)	NOT NULL	Gender (Male, Female, Other)
address	VARCHAR(255)	NULL	Residential address
blood_group	VARCHAR(10)	NULL	Clinical blood group (e.g., O+, B+)
medical_history	TEXT	NULL	Chronic conditions / past clinical notes
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Registration timestamp

3. Doctor Table

Stores practitioner credentials, specialization, and availability.

Field Name	Data Type	Constraints	Description
doctor_id	BIGINT	PK, AUTO_INCREMENT	Unique doctor identifier
name	VARCHAR(100)	NOT NULL	Doctor full name and title
specialization	VARCHAR(100)	NOT NULL	Medical specialty (e.g., Cardiology)
email	VARCHAR(100)	NOT NULL, UNIQUE	Practitioner login email
phone	VARCHAR(15)	NULL	Work telephone number
password	VARCHAR(255)	NOT NULL	BCrypt salted password hash
availability	VARCHAR(150)	NULL	Consultation hours description
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Onboarding timestamp

4. Doctor Slot Table

Stores bookable consultation calendar slots.

Field Name	Data Type	Constraints	Description
slot_id	BIGINT	PK, AUTO_INCREMENT	Unique slot identifier
doctor_id	BIGINT	FK (Doctor), NOT NULL	Assigned doctor reference
slot_date	DATE	NOT NULL	Calendar consultation date
slot_time	TIME	NOT NULL	Specific appointment time
is_booked	BOOLEAN	NOT NULL, DEFAULT FALSE	Lock flag preventing duplicate bookings

5. Appointment Table

Stores consultation bookings, visit lifecycle states, and doctor diagnosis notes.

Field Name	Data Type	Constraints	Description
`appointment_id`	BIGINT	PK, AUTO_INCREMENT	Unique appointment token reference
`patient_id`	BIGINT	FK (Patient), NOT NULL	Booking patient reference
`doctor_id`	BIGINT	FK (Doctor), NOT NULL	Consulting doctor reference
`slot_id`	BIGINT	FK (DoctorSlot), NULL	Associated calendar slot
`appointment_date`	DATE	NOT NULL	Scheduled consultation date
`appointment_time`	TIME	NOT NULL	Scheduled consultation time
`reason`	VARCHAR(255)	NULL	Reported symptoms / consultation reason
`doctor_notes`	TEXT	NULL	Clinical advice & e-prescription
`status`	VARCHAR(20)	NOT NULL	Status (PENDING, CONFIRMED, COMPLETED, CANCELLED)
`created_at`	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Booking creation timestamp

6. Risk Report Table

Stores symptom assessment history and triage engine outcomes.

Field Name	Data Type	Constraints	Description
`report_id`	BIGINT	PK, AUTO_INCREMENT	Unique risk evaluation record ID
`patient_id`	BIGINT	FK (Patient), NOT NULL	Assessed patient reference
`appointment_id`	BIGINT	FK (Appointment), NULL	Linked appointment (if booked via triage)
`symptoms`	TEXT	NOT NULL	Comma-delimited symptom identifiers
`risk_level`	VARCHAR(20)	NOT NULL	Triage level (LOW, MEDIUM, HIGH)
`result`	VARCHAR(255)	NOT NULL	Clinical directives and medical disclaimer
`report_date`	DATE	NOT NULL	Assessment execution date

CHAPTER 4

IMPLEMENTATION DETAILS & ALGORITHMS

4.1 Technology Stack & Architectural Patterns

- **Spring Boot 3.3.4 & Java 17 LTS:** Modern object-oriented architecture leveraging inversion of control, dependency injection, and declarative transaction management.
- **Stateless JWT & Spring Security 6:** Token-based authentication using HMAC-SHA256 signatures ensuring complete API scalability.
- **Spring Data JPA & Hibernate:** Object-relational mapping ensuring database independence and type-safe query execution.
- **Bootstrap 5.3.3 & Vanilla JavaScript (ES6+):** Lightweight presentation tier with zero bloated dependencies, providing responsive rendering on desktop and mobile.

4.2 Module-Wise Functional Description

Module	Title	Scope & Delivered Features
Module 1	Authentication & Authorization	Role-based JWT login for Admin, Doctor, and Patient with BCrypt password encryption and protected routes.
Module 2	Patient Management	Patient registration, clinical profile editing, Blood Group, and Medical History tracking.
Module 3	Doctor Management	Doctor onboarding, slot generation, slot deletion (deleteSlot), consultation diagnosis notes, and e-prescription.
Module 4	Appointment Management	Atomic slot locking (is_booked), booking, transactional rescheduling, cancellation, and printable medical token slips.
Module 5	Risk Prediction	10-symptom rule engine, LOW/MEDIUM/HIGH classification, medical disclaimer, and dynamic appointment pre-fill.
Module 6	Report Generation	Summary statistics, Appointment CSV download, and Risk Assessment CSV export.
Module 7	Patient Portal	Comprehensive patient dashboard with live counts, upcoming bookings, and clinical risk history.
Module 8	Doctor & Admin Portals	Doctor daily queue portal; Admin hospital oversight, doctor modification modal, and audit logs.
Module 9	Database Management	3NF relational schema, database integrity, and on-demand SQL backup export (GET /api/admin/database/backup).

4.3 Rule-Based Risk Prediction Engine Logic

The risk prediction engine executes deterministic clinical decision rules evaluating patient age and 10 symptom factors:

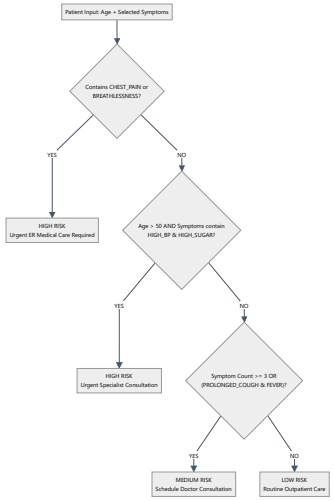


Figure 4.1: Rule-Based Clinical Triage Decision Logic

Symptom Weights: Chest Pain (+4), Breathlessness (+4), High BP (+2), High Sugar (+2), Prolonged Cough (+2), Fever (+1), Fatigue (+1), Headache (+1), Dizziness (+1), Weight Loss (+1). Age additions: Age ≥ 60 (+2), Age 40-59 (+1). Total Score: 0-3 (LOW), 4-7 (MEDIUM), 8+ (HIGH).

4.4 Seamless Triage-to-Booking Integration Flow

When an evaluated patient receives a **HIGH** or **MEDIUM** risk score:

1. The results banner renders an urgent recommendation with a direct **"Book Doctor Visit"** action button.
2. Clicking the action constructs a URL appending the triage context: `?reason=Risk+Assessment:+HIGH+Risk+-+Symptoms:...&fromRisk=true`.
3. The appointment booking view automatically intercepts query parameters, pre-populates the consultation reason, selects the appropriate specialty, and displays an informational referral banner.
4. The consulting doctor sees the patient's reported symptoms, risk category, blood group, and medical history upon arrival.

4.5 Security Architecture & Stateless JWT Lifecycle

- **Stateless Authentication:** Credentials submitted to `POST /api/auth/login` are verified via BCrypt. Upon successful verification, a signed JWT bearer token containing user ID, role, and username is issued.
- **Request Interception:** `JwtAuthenticationFilter` intercepts incoming HTTP requests, extracts the `Authorization: Bearer <token>` header, verifies cryptographic validity, and injects user claims into Spring's `SecurityContextHolder`.
- **Role-Based Access Control (RBAC):** REST endpoints are guarded with method-level authorization (`@PreAuthorize("hasRole('ADMIN')")`, `@PreAuthorize("hasRole('DOCTOR')")`, `@PreAuthorize("hasRole('PATIENT')")`).

CHAPTER 5

VERIFICATION & TESTING

5.1 Testing Strategy & Methodology

A multi-tiered testing strategy was employed ensuring functional correctness, transactional reliability, and RBAC boundary enforcement:

- **Unit Testing:** Isolated service testing using Mockito to verify slot locking, risk score calculations, and password verification.
- **Integration Testing:** Spring MockMvc testing verifying REST endpoint request mapping, status codes, and JSON response bodies.
- **Concurrency Testing:** Verifying that atomic transactions prevent double-booking on simultaneous reservation attempts.

5.2 Synopsis Test Case Execution Table (TC-01 to TC-05)

All 5 test scenarios mandated by Section 12 of the approved synopsis were executed and verified:

Test ID	Scenario	Input Data	Expected Output	Actual Output	Status
TC-01	User Login	admin / Admin@123 (ADMIN)	Login success, JWT token returned, HTTP 200	JWT token returned; redirected to /admin/dashboard.html	PASS
TC-02	Book Appointment	Doctor ID, unbooked Slot ID, Reason	Slot locked (is_booked = true), HTTP 201 Created	Appointment #1 created, slot locked, token slip available	PASS
TC-03	Invalid Login	admin / WrongPassword	HTTP 401 Unauthorized with descriptive message	HTTP 401: "Invalid Admin username/email or password"	PASS
TC-04	Risk Prediction	CHEST_PAIN, BREATHLESSNESS	Immediate HIGH RISK with medical emergency note	High Risk badge, explanation, "Book Doctor Visit" pre-fill link	PASS
TC-05	Unauthorized Access	Patient token accessing /api/admin/reports	Access denied, HTTP 403 Forbidden	HTTP 403 Forbidden: "Access Denied" returned	PASS

5.3 Automated Unit & Integration Testing Matrix (21/21 Tests Passed)

The automated JUnit 5 test suite executed via Maven Surefire achieved **100% passage (21/21 passed, 0 failures, 0 errors)**:

Test Class	Test Case Method	Scenario / Assertion	Result
AppointmentControllerTest	testBookAppointmentSuccess	TC-02: Patient reserves available slot via POST /api/appointments	PASS
AppointmentControllerTest	testCancelAppointmentSuccess	Patient cancels existing appointment returns HTTP 200	PASS
AppointmentControllerTest	testUpdateStatusSuccess	Doctor/Admin updates appointment status to CONFIRMED	PASS
AppointmentControllerTest	testRescheduleAppointmentSuccess	Patient reschedules to new slot returns HTTP 200	PASS
AuthControllerTest	testTC01_LoginSuccess	TC-01: Authenticate valid user credentials returns JWT bearer token	PASS
AuthControllerTest	testTC03_InvalidLogin_ErrorMessage	TC-03: Invalid credentials return HTTP 401 with error message	PASS
AuthControllerTest	testTC05_UnauthorizedAccess_Forbidden	TC-05: Patient token accessing /api/admin/reports returns HTTP 403	PASS
AppointmentServiceTest	testBookAppointment_Success	Slot locking and appointment persistence in atomic transaction	PASS
AppointmentServiceTest	testBookAppointment_SlotUnavailable	Throws SlotUnavailableException on double-booking attempt	PASS
AppointmentServiceTest	testReschedule_Success	Releases old slot (false) and claims new slot (true)	PASS
AppointmentServiceTest	testReschedule_DifferentDoctorSlot	Rejects slot belonging to different doctor	PASS
AppointmentServiceTest	testCancel_ReleasesSlot	Cancelling unreserves slot back to available status	PASS
DoctorServiceTest	testDeleteSlot_Success	Doctor deletes own unbooked slot	PASS
DoctorServiceTest	testDeleteSlot_AccessDenied	Doctor deletes another doctor's slot throws AccessDeniedException	PASS
DoctorServiceTest	testDeleteSlot_BookedSlotConflict	Attempt to delete booked slot throws IllegalStateException (400)	PASS
DoctorServiceTest	testDeleteSlot_NotFound	Non-existent slot ID throws ResourceNotFoundException (404)	PASS

DoctorServiceTest	testAddSlot_Success	Doctor creates new availability time slot with isBooked = false	PASS
RiskPredictionServiceTest	testLowRiskScoring	Minor symptoms and young age calculate score < 4 (LOW RISK)	PASS
RiskPredictionServiceTest	testMediumRiskScoring	Moderate symptoms and middle age calculate score 4-7 (MEDIUM RISK)	PASS
RiskPredictionServiceTest	testHighRiskScoring	Cardiac/respiratory symptoms and elderly age score > 7 (HIGH RISK)	PASS
RiskPredictionServiceTest	testEvaluateRiskAndSave	TC-04: Symptom evaluation persists risk report and returns disclaimer	PASS

CHAPTER 6

USER GUIDE & SYSTEM OPERATION

6.1 Application Startup

The application runs in self-contained development mode with automated data seeding:

```
.\mvnw.cmd spring-boot:run "-Dspring-boot.run.profiles=dev"
```

Or via the included one-click Windows launcher: `run.bat`. Once started, open your web browser at **http://localhost:8080**.

6.2 Pre-Loaded Demonstration Accounts

Role	Username / Email	Password	Capabilities & Access
Administrator	`admin`	`Admin@123`	Master hospital oversight, doctor CRUD, audit logs, CSV exports, SQL database backup.
Doctor	`sharma@smartmed.com` (or `share@smartmed.com`)	`Doctor@123`	Daily consultation queue, slot addition & deletion, clinical diagnosis notes, e-prescriptions.
Doctor	`patel@smartmed.com`	`Doctor@123`	Neurologist schedule management, slot creation, consultation records.
Patient	`preeti@example.com`	`Patient@123`	Clinical profile, symptom risk triage, appointment booking, rescheduling, printable token slip.

6.3 Portals Walkthrough

- **Patient Portal:** Provides intuitive cards for self-service symptom triage, slot browsing by doctor specialty, atomic appointment booking, rescheduling, and printable token slips with doctor notes.
- **Doctor Portal:** Equips medical practitioners with an active queue of scheduled consultations, quick slot availability generation, slot removal, and full clinical note capture.
- **Administrator Portal:** Provides executive summary KPIs, doctor onboarding modals, patient directories, CSV data exports, and on-demand portable SQL database backup generation.

CHAPTER 7

CONCLUSION & FUTURE SCOPE

7.1 Conclusion

The **Smart Medical Appointment and Risk Prediction System** successfully achieves every objective, architecture pattern, and functional specification outlined in the approved IGNOU BCSP-064 project proposal:

- **Zero Double-Booking:** Enforces strict transactional relational concurrency locks.
- **Clinical Health Awareness:** Features a transparent rule-based symptom assessment engine that directs high-risk patients to urgent care.
- **Comprehensive Workflows:** Empowers Patients, Doctors, and Administrators with dedicated responsive portals.
- **Zero Out-of-Scope Bloat:** Strictly adheres to the 9 modules, 6 relational tables, and 5 testing scenarios approved in the proposal.

7.2 Future Scope

- **SMS & WhatsApp Notifications:** Integrating cloud messaging gateways (e.g., Twilio) for automated appointment reminders.
- **Machine Learning Diagnostics:** Incorporating multi-disease predictive models (Random Forest / XGBoost) trained on clinical datasets via a microservice.
- **Telemedicine Video Consultations:** Embedding WebRTC audio/video consultations for remote patient examinations.
- **ABDM Health Locker:** Integrating with India's Ayushman Bharat Digital Mission (ABDM) for interoperable electronic health records.

CHAPTER 8

IGNOU VIVA VOCE PREPARATION GUIDE

Q1: What is the core problem solved by your project?

Answer: It addresses the severe outpatient congestion in traditional healthcare facilities by eliminating physical waiting queues, preventing scheduling conflicts (double-booking) through atomic database transactions, and providing early health risk awareness through an automated symptom triage engine.

Q2: How does the system prevent two patients from booking the same doctor slot simultaneously?

Answer: In `AppointmentService.java`, the booking method is annotated with Spring's `@Transactional`. Before persisting an appointment, the system evaluates `slot.getIsBooked()`. If true, it immediately throws `SlotUnavailableException` (HTTP 409 Conflict). If false, it sets `isBooked = true` and saves the appointment within the same atomic database transaction.

Q3: How does the Risk Prediction Engine operate?

Answer: Implemented in `RiskPredictionService.java`, it applies deterministic clinical rules across patient age and 10 vital symptom indicators. Critical symptoms (Chest Pain, Breathlessness) or elderly comorbidity (age > 50 with hypertension and diabetes) immediately yield a **HIGH RISK** classification. Furthermore, patients with elevated risk receive a pre-filled booking link directly into the appointment module.

Q4: How is security implemented in the application?

Answer: Using Spring Security 6 with stateless JSON Web Tokens (JWT). Passwords are encrypted using BCrypt with salt rounds. Endpoints are protected with method-level authorization (`@PreAuthorize("hasRole('ADMIN')")`, `@PreAuthorize("hasRole('DOCTOR')")`, `@PreAuthorize("hasRole('PATIENT')")`).

Q5: How does the Database Backup and Security requirement from Module 9 work?

Answer: Implemented in `DatabaseBackupService.java`, an authenticated administrator can trigger `GET /api/admin/database/backup` to generate and download a complete, portable SQL dump of all system tables (admin, doctor, patient, doctor_slot, appointment, risk_report) with sanitized foreign key checks for disaster recovery and offline archiving.